PolyStack

TRACK 03 · REASONING ARCHITECTURE

An agent that only patches what is already on disk.

Most coding agents wander the repo and invent SEARCH anchors.

PolyStack Architect locks the work to a real file window first.

1 Locate
Open real code from disk

2 Diagnose
Name the contract there

3 Patch
Edit only if bytes match

PolyStack

THE PROBLEM

Free navigation makes mid-size agents invent the codebase.

They invent SEARCH anchors that are not on disk.

They open the wrong file and still sound sure.

Call-graph contracts quietly disappear.

invented_patch.diff

DOES NOT EXIST

```
<<<<<< SEARCH
def save_form(self, data):
    # hallucinated helper
    return self._magic(data)
=======
def save_form(self, data):
    return self.cleaned(data)
>>>>>> REPLACE

# No file on disk matches SEARCH
# The edit never applies
```

PolyStack

TWO WAYS TO WORK

Wandering invents. Disk-first refuses to lie.

Wandering agent

- Walks the repo freely
- Invented SEARCH anchors
- Patch misses real files
- Sounds confident anyway

PolyStack Architect

- Opens a real disk window first
- SEARCH must match exact bytes
- Edit stays inside that window
- Otherwise: refuse — with a why

PolyStack

STEP 1 · LOCATE

Open the real production file — byte for byte from disk.

The model does not get to pick a convenient file.

We show it a verbatim window. That window is the world.

django/db/models/fields/__init__.py — FROM DISK

```python
class Field:
    def contribute_to_class(self, cls, name):
        self.set_attributes_from_name(name)
        self.model = cls
        cls._meta.add_field(self)
        if self.column:
            setattr(cls, name, self)

# Window copied verbatim — not a paraphrase
```

Locate → Diagnose → Patch

PolyStack

STEP 2 · DIAGNOSE

Understand the rule that must stay true in this window.

What does this code promise its callers?

We name that contract here — before any edit.

No peeking at the answer key. Blind-safe.

What this window must keep true

Rule	Field.column decides attribute install
Caller	contribute_to_class runs at model build
Keep	cls._meta.add_field(self) order
Ignore	Anything outside this window

PolyStack

STEP 3 · PATCH

SEARCH must match the bytes on disk — or the edit never lands.

Replace only inside the Locate window.

If SEARCH is invented, Architect refuses.

apply_patch · disk window — MATCHED

```
<<<<<< SEARCH
        if self.column:
            setattr(cls, name, self)
=======
        if self.column is not None:
            setattr(cls, name, self)
>>>>>> REPLACE

# Exact bytes found in the Locate window
# Patch applies. Node closes clean.
```

Locate → Diagnose → **Patch**

PolyStack

WHAT IT REFUSES

Invented SEARCH never becomes an edit.

No write outside the window. No gold leak. Refusal is first-class.

invented_anchor.diff — REFUSED

```
<<<<<< SEARCH
def _magic_helper(self):
    ...  # never existed on disk
=======
...
>>>>>> REPLACE

Architect: SEARCH does not match disk
Action: no file written
Why: invented anchor outside Locate window
```

PolyStack

SPONSOR TECH · SAME TRAIL

Discovery and memory — not bolted on after the demo.

Zetaris

Finds the right fragments

- → Source discovery for Locate
- → Cuts the repo into real windows
- → Fragments keep provenance

Meterless

Remembers why each step happened

- → Stores the evidence trail
- → You can ask why Locate → Patch
- → Even a refusal keeps the why

PolyStack

ALREADY IN THE ARENA

SWE-bench Lite · 300 real repository bugs

PolyStack Architect v1 is already competing on the official board.

300

Real bugs

Full SWE-bench Lite

~100%

Expected

Our run cleared the pack we submitted

Soon

Official %

Harness still scoring the leaderboard

Earlier public baseline on the same board: Tree-14B · 13 / 300 · 4.33%.

Architect is the reasoning upgrade — disk-first Locate → Diagnose → Patch.

Honesty: official Resolved % is not final until the harness finishes.



PolyStack

Reasoning architecture. Transparency. Creative sovereignty.

Open to commercial partnerships and pilots.

getpolystack.com

github.com/msmourao/20261006_lifegraph_qwen2.5_coder_14b_tree